Welcome to your next lesson: For loops and range(): counting iterations.

I'm preparing your personalized lesson based on what you've already learned, what's up next, and how to make it stick.

Step by step, you're moving forward.

-

Imagine someone asks you to print the numbers 1 to 10 in Python. Typing:

```Python
print(1)
print(2)
print(3)
# ...
print(10)
```

would be super annoying and hard to change later. A for loop lets Python do the repeating for you.

In this lesson we'll use `for` loops together with `range()` to count, skip numbers, and add things up. We'll go one small step at a time, with code you'll try in the editor.

To start simple:

If I write this in Python:

```Python
for i in range(5):
    print(i)
```

What do you expect it will print: will it include the number 5, or stop before 5?

1 2 3 4

✅ Exactly: it prints

```Text
0
1
2
3
4
```

So `range(5)` starts at 0 and stops before 5.

Next tiny step ?

Now look at this:

```Python
for i in range(1, 5):
    print(i)
```

This time we gave two numbers: `range(1, 5)`.

Do you expect it to print 0–4, or 1–4?

1 to 4

Nice, that's right ?

`range(1, 5)` produces: 1, 2, 3, 4 (starts at 1, stops before 5).

So pattern so far:

- `range(stop)` → starts at 0, goes up to `stop - 1`
- `range(start, stop)` → starts at **start**, goes up to `stop - 1`

Let's add one more idea: counting by steps.

Example:

```python
for i in range(1, 10, 2):
    print(i)
```

Here `range(1, 10, 2)` means:

- start at 1
- stop before 10
- go up by 2 each time

Take a guess: what numbers do you think this will print? (Just type them in order.)

3 % & (

Love that you took a swing at it ?

Let's slow it down and walk through `range(1, 10, 2)` step by step:

- Start at 1 → print 1
- Add 2 → now 3 → print 3
- Add 2 → now 5 → print 5
- Add 2 → now 7 → print 7
- Add 2 → now 9 → print 9
- Add 2 → would be 11, but that's past 10, so we stop

So the output is:

```text
1
3
5
7
9
```

These are all the odd numbers between 1 and 9.

Quick check for you ?

Looking at that pattern, what do you think this will print?

```python
for i in range(2, 11, 2):
    print(i)
```

You don’t have to be perfect — just list the numbers you expect.

9 sorry

No need to be sorry at all, Ron — you’re doing fine ? We’re just practicing.

Let’s walk through this one together:

```python
for i in range(2, 11, 2):
    print(i)
```

Step by step:

- Start at 2 → print 2
- Add 2 → now 4 → print 4
- Add 2 → now 6 → print 6
- Add 2 → now 8 → print 8
- Add 2 → now 10 → print 10
- Add 2 → would be 12, but that’s past 11, so we stop

So it prints:

```text
2
4
6
8
10
```

Those are all the even numbers from 2 to 10.

Quick check-in:

Does this “start / add step / stop before the last number” idea make sense so far, or should I show it with a little visual diagram next?

It makes sense

Great ?

Since the idea of **range**(start, stop, step) is clear, let’s now use a loop to add things up.

Imagine you want the sum of the numbers 1 to 5. You could do:

```python
total = 1 + 2 + 3 + 4 + 5
print(total)
```

But with a loop, we do it step by step:

- start with total = 0
- then, for each number, add it into total

In Python:

```python
total = 0
```

```
for i in range(1, 6):
    total = total + i

print(total)
```

We call this pattern an accumulator: a variable (`total`) that keeps a running sum.

Before we try this in the editor:

Step-by-step in your head, does the idea of “start at 0, then keep adding i each time” feel okay?

Yes, that makes sense

Nice ?

Let’s have you use this idea in the code editor now.

Here’s what I’d like you to do in the editor (not in chat):

1. Set up an accumulator:
   `total = 0`
2. Write a `for` loop that goes from 1 to 5 (inclusive) using `range(...)`.
3. Inside the loop, update `total` by adding the loop variable each time.
4. After the loop finishes, `print(total)`.

Then hit Run to check it, and when you’re happy, hit Submit code so I can see it and give feedback.

Python
```
total = 0
for i in range(1, 5):
    total = total + i
print(total)
```

Nice work, Ron ?

Your code:

Python
```
total = 0
for i in range(1, 5):
    total = total + i
print(total)
```

is almost what I asked for — it correctly adds the numbers 1 + 2 + 3 + 4 and prints 10.

The only tiny mismatch: I asked for 1 to 5 (inclusive), and `range(1, 5)` stops before 5. To include 5, it needs to be `range(1, 6)`.

Let’s do one more small challenge:

In the editor, change your code so it:

- uses `range(1, 11)`

- still uses `total` the same way
- prints the sum of numbers from 1 to 10

Then Run, and when it looks good, hit Submit code again.

```python
total = 0
for i in range(1, 11):
    total = total + i
print(total)
```

Perfect ✅

Your code:

```python
total = 0
for i in range(1, 11):
    total = total + i
print(total)
```

- uses `range(1, 11)` → numbers 1 through 10
- correctly accumulates into `total`
- prints 55, which is the right sum ?

Last concept for this lesson: countdown with `range()`

To count down from 5 to 1, we use a negative step:

```python
for i in range(5, 0, -1):
    print(i)
```

This means:

- start at 5
- stop before 0
- go down by 1 each time (`-1`)

So it prints: `5 4 3 2 1` (each on its own line).

Make sense how that countdown `range(5, 0, -1)` works, or want me to show it step by step like we did with the sums?

Yes, that makes sense

What do you think `range(2, 9, 2)` gives you? Take your time with the question below ?

What does `range(2, 9, 2)` produce?

A.

2 3 4 5 6 7 8

B.

2 4 6 8

C.

3 5 7 9

D.

2 4 6 8 10

Nice job, Ron — you nailed it ?

`range(2, 9, 2)` means:

- start at 2
- stop before 9
- step by 2 each time

So the sequence is: 2, 4, 6, 8 — exactly what you picked.

You've now used:

- **range**(stop)
- **range**(start, stop)
- **range**(start, stop, step)
- an accumulator (`total`) to add things up
- even/odd sequences and a countdown

You've hit all the goals for this lesson ✅

Ready to wrap up this lesson and move on to the next one?

Yes, I'm ready

Awesome work today, Ron ?

You:

- Used `for` loops with all three **range**() forms: **range**(stop), **range**(start, stop), **range**(start, stop, step)
- Built an accumulator to sum numbers (like 1–10 → 55)
- Generated even sequences and understood countdowns with a negative step

These are core Python skills you'll use all the time as you go deeper.

When you're ready, you can head to the Next Lesson in the Learn tab, and I'll keep going with you there ?